Updated Implementation Guide: Tracking Qualified Leads from VSL to Purchase

This guide provides a complete step-by-step process for setting up a pipeline that tracks Facebook conversions through your specific funnel: GHL VSL → Typeform survey → Calendly → Event Attendance → Payment. This will enable you to optimize your Facebook ads for multiple conversion events, from initial booking to final purchase.

Section 1: Updated Pipeline Design

This section outlines the complete architecture of your specific tracking pipeline, from the VSL page to the final purchase.

Updated Pipeline Design for VSL → Typeform → Calendly Funnel

Funnel Structure Overview

Customer Journey Flow

```
Facebook Ad Click (fbclid captured)
        ↓
GHL VSL Page (PageView pixel event)
        ↓
Typeform Survey
        ↓
    ┌──────────────────────────────┐
    │   Survey Logic   │
    └──────────────────────────────┘
        ↓
    ┌──────────────────┬──────────────────┐
    │ Qualified  │ Unqualified │
    │   Route    │   Route    │
    └──────────────────┴──────────────────┘
        ↓          ↓
  Calendly Page  Thank You Page
```

```
(Schedule event) (No tracking)
        ↓
Appointment Booked
(Lead in GHL pipeline)
        ↓
Event Day - Show Up
(CompleteRegistration event)
        ↓
Payment Made
(Purchase event)
```

Tracking Points and Pixel Events

1. VSL Page (GoHighLevel)

What happens:
- User lands on VSL page from Facebook ad
- Facebook Pixel fires "PageView" event
- fbclid parameter is captured and stored
- User watches VSL and clicks to survey

Pixel Event: `PageView`
Data Captured: fbclid, page URL, timestamp

2. Typeform Survey

What happens:
- User completes qualification survey
- Survey logic determines qualification
- Qualified users → Calendly page
- Unqualified users → Thank you page

Pixel Event: None (Typeform handles routing)
Data Captured: Survey responses, qualification status

3. Qualified Thank You Page (After Calendly Booking)

What happens:
- User successfully books appointment via Calendly
- Redirected to qualified thank you page
- Facebook Pixel fires "Schedule" event
- Lead data is sent to GoHighLevel

Pixel Event: `Schedule` (Custom Event)
Data Captured: Appointment details, contact info

4. Event Attendance (Show Up)

What happens:
- User attends the booked appointment
- Attendance is confirmed manually or automatically
- Facebook Pixel fires "CompleteRegistration" event
- This is your qualified lead conversion

Pixel Event: `CompleteRegistration`
Data Captured: Attendance confirmation, event details

5. Payment Completion (Pay in Full)

What happens:
- User makes full payment for service/product
- Facebook Pixel fires "Purchase" event
- Revenue and transaction details tracked

Pixel Event: `Purchase`
Data Captured: Transaction amount, payment details

Technical Implementation Requirements

Facebook Pixel Setup

Required Events:
1. `PageView` - VSL page visits
2. `Schedule` - Appointment bookings (custom event)
3. `CompleteRegistration` - Event attendance
4. `Purchase` - Payment completion

GoHighLevel Pipeline Stages

Updated Pipeline:
1. `New Lead` - Initial VSL visitor
2. `Survey Completed - Qualified` - Passed Typeform qualification
3. `Appointment Booked` - Scheduled via Calendly
4. `Attended Event - Qualified Lead` - Showed up (triggers CompleteRegistration)

5. `Paid in Full` - Made payment (triggers Purchase)
6. `No Show` - Booked but didn't attend

Data Flow Integration Points

Point 1: VSL to Typeform

- Capture fbclid on VSL page
- Pass fbclid to Typeform via URL parameter
- Store in browser localStorage as backup

Point 2: Typeform to Calendly

- Pass qualification status and fbclid to Calendly
- Only qualified leads reach Calendly
- Unqualified leads see thank you page (no tracking)

Point 3: Calendly to GoHighLevel

- Calendly webhook sends booking data to GHL
- Include fbclid in the webhook payload
- Create contact with "Appointment Booked" stage

Point 4: Attendance Tracking

- Manual or automated check-in process
- Update pipeline stage to "Attended Event"
- Trigger CompleteRegistration pixel event via Zapier

Point 5: Payment Tracking

- Payment system integration with GHL
- Update pipeline stage to "Paid in Full"
- Trigger Purchase pixel event via Zapier

Updated Zapier Workflows

Workflow 1: Attendance Tracking

Trigger: GHL pipeline stage change to "Attended Event - Qualified Lead"
Action: Send CompleteRegistration event to Facebook Conversions API

Workflow 2: Payment Tracking

Trigger: GHL pipeline stage change to "Paid in Full"
Action: Send Purchase event to Facebook Conversions API

Key Differences from Original Design

1. Multiple Conversion Events

- Original: Single "Purchase" event for attendance
- Updated: CompleteRegistration for attendance, Purchase for payment

2. Qualification Filtering

- Original: All bookings tracked
- Updated: Only qualified leads from Typeform reach tracking

3. Multi-Stage Funnel

- Original: Direct booking form
- Updated: VSL → Survey → Calendly → Attendance → Payment

4. Event Value Hierarchy

- Schedule: Low value (booking intent)
- CompleteRegistration: Medium value (qualified attendance)
- Purchase: High value (revenue generation)

Attribution and Optimization Strategy

Facebook Optimization Goals

1. **Volume Campaigns:** Optimize for PageView (VSL traffic)
2. **Quality Campaigns:** Optimize for Schedule (qualified bookings)
3. **Revenue Campaigns:** Optimize for CompleteRegistration (attendance)
4. **ROI Campaigns:** Optimize for Purchase (payments)

Attribution Windows

- PageView to Schedule: 1-7 days
- Schedule to CompleteRegistration: 1-30 days

- CompleteRegistration to Purchase: 1-7 days

Value Assignment Strategy

```
Event Values:
- PageView: $0 (awareness)
- Schedule: $25 (qualified booking)
- CompleteRegistration: $100 (attended event)
- Purchase: Actual transaction amount
```

This updated design provides much more granular tracking and allows Facebook to optimize at different stages of your funnel, from initial interest to final payment.

Section 2: Updated GoHighLevel Setup

This section provides detailed instructions for configuring GoHighLevel to support your specific VSL → Typeform → Calendly funnel.

Updated GoHighLevel Setup for VSL → Typeform → Calendly Funnel

Step 1: Create Custom Fields for Enhanced Tracking

1.1 Core Tracking Fields

Create these custom fields in GoHighLevel:

Facebook Click ID Field:
- **Field Name:** `Facebook Click ID`
- **Field Type:** `Text`
- **Field Key:** `facebook_click_id`
- **Required:** No
- **Description:** "Facebook click ID for conversion tracking"

Qualification Status Field:
- **Field Name:** `Qualification Status`
- **Field Type:** `Dropdown`

- **Options:** `Qualified` , `Unqualified` , `Pending Review`
- **Field Key:** `qualification_status`

Survey Responses Field:
- **Field Name:** `Survey Responses`
- **Field Type:** `Text Area`
- **Field Key:** `survey_responses`
- **Description:** "Typeform survey responses in JSON format"

Appointment Details Field:
- **Field Name:** `Appointment Details`
- **Field Type:** `Text Area`
- **Field Key:** `appointment_details`
- **Description:** "Calendly appointment information"

Event Attendance Field:
- **Field Name:** `Event Attendance`
- **Field Type:** `Dropdown`
- **Options:** `Scheduled` , `Confirmed` , `Attended` , `No Show` , `Cancelled`
- **Field Key:** `event_attendance`

Payment Status Field:
- **Field Name:** `Payment Status`
- **Field Type:** `Dropdown`
- **Options:** `Not Started` , `Partial` , `Paid in Full` , `Refunded`
- **Field Key:** `payment_status`

Revenue Amount Field:
- **Field Name:** `Revenue Amount`
- **Field Type:** `Number`
- **Field Key:** `revenue_amount`
- **Description:** "Total revenue from this lead"

Step 2: Configure Updated Pipeline Stages

2.1 Create New Pipeline: "VSL to Purchase Funnel"

1. Go to **CRM** → **Pipelines**
2. Click **"Add Pipeline"**
3. **Pipeline Name:** "VSL to Purchase Funnel"

2.2 Set Up Pipeline Stages

Configure these stages in order:

Stage 1: VSL Visitor
- **Stage Name:** `VSL Visitor`
- **Color:** Light Blue
- **Description:** Visited VSL page, PageView tracked

Stage 2: Survey Completed - Qualified
- **Stage Name:** `Survey Completed - Qualified`
- **Color:** Blue
- **Description:** Passed Typeform qualification

Stage 3: Appointment Booked
- **Stage Name:** `Appointment Booked`
- **Color:** Yellow
- **Description:** Scheduled appointment via Calendly (Schedule event)

Stage 4: Attended Event - Qualified Lead
- **Stage Name:** `Attended Event - Qualified Lead`
- **Color:** Green
- **Description:** Showed up to appointment (CompleteRegistration event)

Stage 5: Paid in Full
- **Stage Name:** `Paid in Full`
- **Color:** Dark Green
- **Description:** Made full payment (Purchase event)

Stage 6: No Show
- **Stage Name:** `No Show`
- **Color:** Red
- **Description:** Booked but didn't attend

Stage 7: Unqualified
- **Stage Name:** `Unqualified`
- **Color:** Gray
- **Description:** Did not pass Typeform qualification

Step 3: VSL Page Setup in GoHighLevel

3.1 Create VSL Page

1. Go to **Sites** → **Funnels**
2. Create new funnel or edit existing
3. Add VSL page as first step

3.2 Add Facebook Pixel to VSL Page

In the VSL page settings → Custom Code → Header:

```
<!-- Facebook Pixel Code -->
<script>
!function(f,b,e,v,n,t,s)
{if(f.fbq)return;n=f.fbq=function(){n.callMethod?
n.callMethod.apply(n,arguments):n.queue.push(arguments)};
if(!f._fbq)f._fbq=n;n.push=n;n.loaded=!0;n.version='2.0';
n.queue=[];t=b.createElement(e);t.async=!0;
t.src=v;s=b.getElementsByTagName(e)[0];
s.parentNode.insertBefore(t,s)}(window, document,'script',
'https://connect.facebook.net/en_US/fbevents.js');

fbq('init', 'YOUR_PIXEL_ID');
fbq('track', 'PageView');
</script>

<!-- Facebook Click ID Capture -->
<script>
// Function to get URL parameter
function getUrlParameter(name) {
    name = name.replace(/[\[]/, '\\[
').replace(/[\]]/, '\\]
');
    var regex = new RegExp('[\\?&]' + name + '=([^&#]*)
');
    var results = regex.exec(location.search);
    return results === null ? '' : decodeURIComponent(results[1].replace(/\+/g, ' '));
}

// Capture and store Facebook Click ID
document.addEventListener('DOMContentLoaded', function() {
    var fbclid = getUrlParameter('fbclid');

    if (fbclid) {
        // Store in localStorage
        localStorage.setItem('fbclid', fbclid);
```

```
        // Store in sessionStorage as backup
        sessionStorage.setItem('fbclid', fbclid);

        console.log('Facebook Click ID captured:', fbclid);
    }
});

// Function to get stored fbclid
function getStoredFbclid() {
    return localStorage.getItem('fbclid') || sessionStorage.getItem('fbclid') || '';
}
</script>
```

3.3 Add CTA Button to Typeform

Update your VSL CTA button to pass fbclid to Typeform:

```
 <script>
function goToSurvey() {
    var fbclid = getStoredFbclid();
    var typeformUrl = 'https://your-typeform-url.typeform.com/to/YOUR_FORM_ID';

    if (fbclid) {
        typeformUrl += '?fbclid=' + encodeURIComponent(fbclid);
    }

    window.location.href = typeformUrl;
}
</script>

<button onclick="goToSurvey()" class="cta-button">
    Take the Survey
</button>
```

Step 4: Typeform Integration Setup

4.1 Configure Typeform Hidden Fields

In your Typeform, add these hidden fields:
- `fbclid` - Facebook Click ID
- `source` - Traffic source (set to "facebook")

4.2 Set Up Typeform Logic Jumps

Configure logic to route based on qualification:

For Qualified Leads:
- Redirect to: `https://calendly.com/your-calendar?fbclid={{hidden:fbclid}}`

For Unqualified Leads:
- Redirect to: Your unqualified thank you page (no tracking)

4.3 Typeform Webhook to GoHighLevel

1. In Typeform, go to **Connect** → **Webhooks**
2. Add webhook URL: `https://services.leadconnectorhq.com/hooks/[YOUR_GHL_WEBHOOK]`
3. Configure to send all responses

Step 5: Calendly Integration

5.1 Calendly Webhook Setup

1. In Calendly, go to **Integrations** → **Webhooks**
2. Add webhook URL pointing to GoHighLevel
3. Subscribe to "Invitee Created" events

5.2 Create Calendly Thank You Page

Create a thank you page that:
1. Fires the "Schedule" pixel event
2. Captures the fbclid from URL parameter
3. Sends data to GoHighLevel

Thank You Page Code:

```
<!-- Facebook Pixel Schedule Event -->
<script>
document.addEventListener('DOMContentLoaded', function() {
  // Get fbclid from URL
  var fbclid = getUrlParameter('fbclid');

  if (fbclid) {
    // Fire Schedule event
    fbq('track', 'Schedule', {
      content_name: 'Appointment Booking',
      value: 25.00,
      currency: 'USD'
    });
```

```
        console.log('Schedule event fired for fbclid:', fbclid);
    }
});
</script>
```

Step 6: Automation Workflows

6.1 Typeform Response Automation

Trigger: Webhook from Typeform
Actions:
1. Create/Update contact in GoHighLevel
2. Set custom field "Facebook Click ID" from webhook data
3. Set custom field "Survey Responses" with form data
4. Set custom field "Qualification Status" based on responses
5. Move to appropriate pipeline stage:
   - Qualified → "Survey Completed - Qualified"
   - Unqualified → "Unqualified"

6.2 Calendly Booking Automation

Trigger: Webhook from Calendly
Actions:
1. Find contact by email in GoHighLevel
2. Set custom field "Appointment Details" with booking info
3. Set custom field "Event Attendance" to "Scheduled"
4. Move to pipeline stage "Appointment Booked"

6.3 Attendance Confirmation Automation

Trigger: Manual pipeline stage change to "Attended Event - Qualified Lead"
Actions:
1. Set custom field "Event Attendance" to "Attended"
2. Trigger webhook to Zapier for CompleteRegistration event

6.4 Payment Completion Automation

Trigger: Manual pipeline stage change to "Paid in Full"
Actions:
1. Set custom field "Payment Status" to "Paid in Full"

2. Update custom field "Revenue Amount"
3. Trigger webhook to Zapier for Purchase event

Step 7: Webhook Configuration for Zapier

7.1 Set Up Multiple Webhooks

Create separate webhooks for different events:

Webhook 1: Attendance Tracking
- **Name:** "CompleteRegistration Event"
- **Trigger:** Pipeline stage change to "Attended Event - Qualified Lead"
- **URL:** [Zapier webhook URL for CompleteRegistration]

Webhook 2: Payment Tracking
- **Name:** "Purchase Event"
- **Trigger:** Pipeline stage change to "Paid in Full"
- **URL:** [Zapier webhook URL for Purchase]

7.2 Webhook Data Configuration

Ensure webhooks send:
- Contact ID and details
- Facebook Click ID
- Pipeline stage information
- Custom field values
- Timestamp of change

Step 8: Testing and Validation

8.1 End-to-End Test Checklist

- [ ] VSL page captures fbclid correctly
- [ ] Typeform receives fbclid parameter
- [ ] Qualified leads reach Calendly with fbclid
- [ ] Unqualified leads see thank you page (no tracking)
- [ ] Calendly booking creates contact in GHL
- [ ] Schedule event fires on Calendly thank you page
- [ ] Manual attendance marking triggers webhook
- [ ] Payment completion triggers webhook

8.2 Data Validation

- [ ] All custom fields populate correctly
- [ ] Pipeline stages update automatically
- [ ] Facebook Click ID persists through entire funnel
- [ ] Webhook payloads contain required data

This updated setup provides comprehensive tracking through your entire VSL → Typeform → Calendly → Attendance → Payment funnel, with proper Facebook pixel events at each stage.

Section 3: Updated Facebook Pixel Tracking

This section provides detailed instructions for setting up Facebook pixel tracking for your multiple conversion events.

Updated Facebook Pixel Tracking for Multiple Conversion Events

Overview of Tracking Events

Your funnel requires tracking four distinct Facebook pixel events:
1. **PageView** - VSL page visits
2. **Schedule** - Qualified appointment bookings
3. **CompleteRegistration** - Event attendance (qualified leads)
4. **Purchase** - Payment completion

Step 1: Facebook Events Manager Configuration

1.1 Set Up Custom Conversions

1. Go to **Events Manager** → **Custom Conversions**
2. Create custom conversions for each event:

Custom Conversion 1: VSL Views
- **Name:** "VSL Page Views"
- **Data Source:** Your Pixel

- **Event:** PageView
- **URL Contains:** `/vsl` or your VSL page URL
- **Conversion Value:** $0

Custom Conversion 2: Qualified Bookings
- **Name:** "Qualified Appointment Bookings"
- **Data Source:** Your Pixel
- **Event:** Schedule (Custom Event)
- **URL Contains:** `/thank-you-qualified` or your Calendly thank you page
- **Conversion Value:** $25

Custom Conversion 3: Event Attendance
- **Name:** "Event Attendance - Qualified Leads"
- **Data Source:** Your Pixel + Conversions API
- **Event:** CompleteRegistration
- **Conversion Value:** $100

Custom Conversion 4: Full Payment
- **Name:** "Payment Completed"
- **Data Source:** Your Pixel + Conversions API
- **Event:** Purchase
- **Conversion Value:** Dynamic (actual payment amount)

1.2 Configure Conversion Windows

Set appropriate attribution windows for each event:
- **PageView:** 1-day view, 7-day click
- **Schedule:** 1-day view, 7-day click
- **CompleteRegistration:** 7-day view, 28-day click
- **Purchase:** 7-day view, 28-day click

Step 2: Pixel Implementation by Page

2.1 VSL Page Pixel Code

```
<!-- Facebook Pixel Base Code -->
<script>
!function(f,b,e,v,n,t,s)
{if(f.fbq)return;n=f.fbq=function(){n.callMethod?
n.callMethod.apply(n,arguments):n.queue.push(arguments)};
if(!f._fbq)f._fbq=n;n.push=n;n.loaded=!0;n.version=\'2.0\';
n.queue=[];t=b.createElement(e);t.async=!0;
t.src=v;s=b.getElementsByTagName(e)[0];
```

```
s.parentNode.insertBefore(t,s)}(window, document,\'script\',
\'https://connect.facebook.net/en_US/fbevents.js\');

fbq(\'init\', \'YOUR_PIXEL_ID\');
fbq(\'track\', \'PageView\');
</script>

<!-- Enhanced PageView Tracking -->
<script>
document.addEventListener(\'DOMContentLoaded\', function() {
    // Track VSL specific page view
    fbq(\'track\', \'PageView\', {
        content_name: \'VSL Page\',
        content_category: \'Video Sales Letter\',
        source: \'facebook\'
    });

    // Capture and store fbclid
    var fbclid = getUrlParameter(\'fbclid\');
    if (fbclid) {
        localStorage.setItem(\'fbclid\', fbclid);
        sessionStorage.setItem(\'fbclid\', fbclid);

        // Track with fbclid for better attribution
        fbq(\'track\', \'PageView\', {
            content_name: \'VSL Page\',
            fb_click_id: fbclid
        });
    }
});

// Video engagement tracking (optional)
function trackVideoProgress(percentage) {
    fbq(\'trackCustom\', \'VideoProgress\', {
        content_name: \'VSL Video\',
        progress: percentage,
        value: percentage * 0.25 // Assign incremental value
    });
}
</script>
```

2.2 Calendly Thank You Page Pixel Code

```
 <!-- Facebook Pixel Base Code -->
<script>
!function(f,b,e,v,n,t,s)
{if(f.fbq)return;n=f.fbq=function(){n.callMethod?
n.callMethod.apply(n,arguments):n.queue.push(arguments)};
if(!f._fbq)f._fbq=n;n.push=n;n.loaded=!0;n.version=\'2.0\';
n.queue=[];t=b.createElement(e);t.async=!0;
```

```
t.src=v;s=b.getElementsByTagName(e)[0];
s.parentNode.insertBefore(t,s)}(window, document,\'script\',
\'https://connect.facebook.net/en_US/fbevents.js\');

fbq(\'init\', \'YOUR_PIXEL_ID\');
fbq(\'track\', \'PageView\');
</script>

<!-- Schedule Event Tracking -->
<script>
document.addEventListener(\'DOMContentLoaded\', function() {
    // Get fbclid from URL parameter
    var fbclid = getUrlParameter(\'fbclid\') | |
                localStorage.getItem(\'fbclid\') | |
                sessionStorage.getItem(\'fbclid\');

    if (fbclid) {
        // Fire Schedule event for qualified booking
        fbq(\'track\', \'Schedule\', {
            content_name: \'Qualified Appointment Booking\',
            content_category: \'Appointment\',
            value: 25.00,
            currency: \'USD\',
            fb_click_id: fbclid
        });

        console.log(\'Schedule event fired for fbclid:\', fbclid);

        // Store booking timestamp for later use
        localStorage.setItem(\'booking_timestamp\', Date.now());
    } else {
        console.warn(\'No Facebook Click ID found for Schedule event\');
    }
});

// Function to get URL parameter
function getUrlParameter(name) {
    name = name.replace(/[\[]/, \'\\[\').replace(/[\]]/, \'\\]\');
    var regex = new RegExp(\'[\\?&]\' + name + \'=([^&#]*)\');
    var results = regex.exec(location.search);
    return results === null ? \'\' : decodeURIComponent(results[1].replace(/\+/g, \'
\'));
}
</script>
```

Step 3: Conversions API Event Structure

3.1 CompleteRegistration Event (Attendance)

```
{
 "data": [
   {
     "event_name": "CompleteRegistration",
     "event_time": 1641234567,
     "action_source": "website",
     "event_source_url": "https://yoursite.com/appointment",
     "user_data": {
       "em": ["hashed_email"],
       "ph": ["hashed_phone"],
       "fn": ["hashed_first_name"],
       "ln": ["hashed_last_name"],
       "fbc": "fb.1.1641234567890.AbCdEfGhIjKlMnOpQrStUvWxYz1234567890"
     },
     "custom_data": {
       "content_name": "Event Attendance",
       "content_category": "Qualified Lead",
       "value": 100.00,
       "currency": "USD",
       "status": "attended"
     },
     "event_id": "attendance_{{contact_id}}_{{timestamp}}"
   }
 ]
}
```

3.2 Purchase Event (Payment)

```
{
 "data": [
   {
     "event_name": "Purchase",
     "event_time": 1641234567,
     "action_source": "website",
     "event_source_url": "https://yoursite.com/payment",
     "user_data": {
       "em": ["hashed_email"],
       "ph": ["hashed_phone"],
       "fn": ["hashed_first_name"],
       "ln": ["hashed_last_name"],
       "fbc": "fb.1.1641234567890.AbCdEfGhIjKlMnOpQrStUvWxYz1234567890"
     },
     "custom_data": {
       "content_name": "Full Payment",
```

```
      "content_category": "Revenue",
      "value": "{{actual_payment_amount}}",
      "currency": "USD",
      "content_ids": ["{{product_id}}"],
      "content_type": "product",
      "num_items": 1
    },
    "event_id": "purchase_{{contact_id}}_{{timestamp}}"
  }
 ]
}
```

Step 4: Event Deduplication Strategy

4.1 Event ID Generation

Use consistent event ID patterns to prevent duplicates:

```
// For CompleteRegistration
const attendanceEventId = `attendance_${contactId}_${eventDate}_${timestamp}`;

// For Purchase
const purchaseEventId = `purchase_${contactId}_${transactionId}_${timestamp}`;
```

4.2 Pixel vs Conversions API Deduplication

Ensure both Pixel and Conversions API use the same event IDs:

Client-side (Pixel):

```
fbq(\'track\', \'CompleteRegistration\', {
  // ... other parameters
}, {
  eventID: \'attendance_12345_20240903_1693747200\'
});
```

Server-side (Conversions API):

```
{
 "event_id": "attendance_12345_20240903_1693747200"
}
```

Step 5: Advanced Tracking Features

5.1 Enhanced Audience Building

Create custom audiences based on each event:

Audience 1: VSL Viewers
- **Source:** PageView events on VSL page
- **Retention:** 180 days
- **Use:** Retargeting warm traffic

Audience 2: Qualified Leads
- **Source:** Schedule events
- **Retention:** 365 days
- **Use:** Lookalike audiences for targeting

Audience 3: Event Attendees
- **Source:** CompleteRegistration events
- **Retention:** 365 days
- **Use:** High-value lookalike audiences

Audience 4: Customers
- **Source:** Purchase events
- **Retention:** 365 days
- **Use:** Customer retention and upsell campaigns

5.2 Dynamic Event Values

Implement dynamic value assignment based on lead quality:

```
// Calculate dynamic value for CompleteRegistration
function calculateAttendanceValue(leadData) {
  let baseValue = 100;

  // Increase value for high-intent leads
  if (leadData.surveyScore > 8) baseValue *= 1.5;
  if (leadData.bookingSpeed < 24) baseValue *= 1.2; // Booked within 24 hours
  if (leadData.previousCustomer) baseValue *= 2;

  return Math.round(baseValue);
}

// Fire event with dynamic value
fbq(\'track\', \'CompleteRegistration\', {
  value: calculateAttendanceValue(leadData),
```

```
    currency: \'USD\'
  });
```

Step 6: Campaign Optimization Strategy

6.1 Campaign Structure by Event

Create separate campaigns optimized for different events:

Campaign 1: Awareness (PageView)
- **Objective:** Traffic
- **Optimization:** PageView
- **Audience:** Broad targeting
- **Budget:** 30% of total

Campaign 2: Qualified Leads (Schedule)
- **Objective:** Conversions
- **Optimization:** Schedule events
- **Audience:** VSL viewers (warm)
- **Budget:** 40% of total

Campaign 3: Quality Attendance (CompleteRegistration)
- **Objective:** Conversions
- **Optimization:** CompleteRegistration
- **Audience:** Qualified leads lookalike
- **Budget:** 25% of total

Campaign 4: Revenue (Purchase)
- **Objective:** Conversions
- **Optimization:** Purchase
- **Audience:** Event attendees lookalike
- **Budget:** 5% of total

6.2 Bidding Strategy

Use value-based bidding with proper event values:
- PageView: $0 (awareness only)
- Schedule: $25 (qualified booking value)
- CompleteRegistration: $100 (attendance value)
- Purchase: Actual revenue amount

Step 7: Testing and Validation

7.1 Pixel Testing Checklist

- [ ] Facebook Pixel Helper shows all events firing
- [ ] PageView events appear in Events Manager
- [ ] Schedule events fire on Calendly thank you page
- [ ] Event parameters include proper fbclid
- [ ] Custom parameters populate correctly

7.2 Conversions API Testing

- [ ] Test events appear in Events Manager
- [ ] Event deduplication works correctly
- [ ] Attribution matches between Pixel and API
- [ ] Event timing is within attribution windows

7.3 End-to-End Attribution Test

1. Click Facebook ad with test fbclid
2. Complete VSL → Typeform → Calendly flow
3. Verify Schedule event fires with correct fbclid
4. Mark attendance in GoHighLevel
5. Verify CompleteRegistration event via Conversions API
6. Complete payment process
7. Verify Purchase event with revenue data

This comprehensive tracking setup ensures accurate attribution and optimization across your entire funnel, from initial VSL view to final payment.

Section 4: Updated Zapier Workflows

This section provides detailed instructions for setting up the two Zapier workflows required to track your multiple conversion events.

Updated Zapier Workflows for Multiple Event Types

Overview

You\'ll need two separate Zapier workflows to handle the different conversion events:
1. **Workflow 1:** CompleteRegistration (Event Attendance)
2. **Workflow 2:** Purchase (Payment Completion)

Workflow 1: CompleteRegistration Event (Event Attendance)

1.1 Create New Zap

1. Log into Zapier
2. Click **"Create Zap"**
3. Name: "GHL Attendance → Facebook CompleteRegistration"

1.2 Set Up Trigger - Webhooks by Zapier

Choose Trigger App: Webhooks by Zapier
Event: Catch Hook
Webhook URL: Copy the provided URL

1.3 Configure GoHighLevel Webhook

In GoHighLevel:
1. Go to **Settings** → **Integrations** → **Webhooks**
2. Create webhook: "CompleteRegistration Event"
3. **Trigger:** Pipeline stage change to "Attended Event - Qualified Lead"
4. **URL:** Paste Zapier webhook URL

1.4 Add Filter Step

Filter Conditions (ALL must be true):
- Pipeline stage contains "Attended Event - Qualified Lead"
- Facebook Click ID exists

- Email exists
- Event attendance = "Attended"

1.5 Add Code Step for CompleteRegistration Processing

Input Data:
- `email` : Contact email
- `phone` : Contact phone
- `firstName` : Contact first name
- `lastName` : Contact last name
- `facebookClickId` : Facebook Click ID
- `contactId` : GoHighLevel contact ID
- `eventName` : Event/appointment name
- `pixelId` : Your Facebook Pixel ID

JavaScript Code:

```
 const crypto = require(\'crypto\');

function hashData(data) {
  if (!data) return null;
  return
crypto.createHash(\'sha256\').update(data.toLowerCase().trim()).digest(\'hex\');
}

function normalizePhone(phone) {
  if (!phone) return null;
  let normalized = phone.replace(/\D/g, \'\');
  if (normalized.length === 10) {
    normalized = \'1\' + normalized;
  }
  return normalized;
}

function formatFacebookClickId(fbclid) {
  if (!fbclid) return null;

  if (fbclid.startsWith(\'fb.1.\')) {
    return fbclid;
  }

  const timestamp = Math.floor(Date.now() / 1000);
  return `fb.1.${timestamp}.${fbclid}`;
}

// Calculate dynamic value based on lead quality
function calculateAttendanceValue(inputData) {
```

```
  let baseValue = 100.00;

  // Increase value for premium events
  if (inputData.eventName &&
inputData.eventName.toLowerCase().includes(\'premium\')) {
    baseValue *= 1.5;
  }

  // Increase value for VIP events
  if (inputData.eventName &&
inputData.eventName.toLowerCase().includes(\'vip\')) {
    baseValue *= 2;
  }

  return baseValue;
}

// Process the input data
const processedData = {
  hashedEmail: hashData(inputData.email),
  hashedPhone: hashData(normalizePhone(inputData.phone)),
  hashedFirstName: hashData(inputData.firstName),
  hashedLastName: hashData(inputData.lastName),
  formattedFbc: formatFacebookClickId(inputData.facebookClickId),
  eventTime: Math.floor(Date.now() / 1000),
  eventId: `attendance_${inputData.contactId}_${Date.now()}`,
  pixelId: inputData.pixelId,
  eventValue: calculateAttendanceValue(inputData),
  eventName: inputData.eventName || \'Event Attendance\',
  originalEmail: inputData.email,
  originalPhone: inputData.phone
};

// Validate required data
if (!processedData.hashedEmail || !processedData.formattedFbc) {
  throw new Error(\'Missing required data: email or Facebook Click ID\');
}

output = [{processedData}];
```

1.6 Add HTTP Request to Facebook Conversions API

URL: `https://graph.facebook.com/v18.0/{{processedData.pixelId}}/events`

Method: POST

Headers:

```
 {
  "Content-Type": "application/json",
```

```
  "Authorization": "Bearer YOUR_FACEBOOK_ACCESS_TOKEN"
}
```

Data (JSON):

```
{
 "data": [
  {
   "event_name": "CompleteRegistration",
   "event_time": "{{processedData.eventTime}}",
   "action_source": "website",
   "event_source_url": "https://yoursite.com/appointment",
   "user_data": {
    "em": ["{{processedData.hashedEmail}}"],
    "ph": ["{{processedData.hashedPhone}}"],
    "fn": ["{{processedData.hashedFirstName}}"],
    "ln": ["{{processedData.hashedLastName}}"],
    "fbc": "{{processedData.formattedFbc}}"
   },
   "custom_data": {
    "content_name": "{{processedData.eventName}}",
    "content_category": "Event Attendance",
    "value": "{{processedData.eventValue}}",
    "currency": "USD",
    "status": "attended"
   },
   "event_id": "{{processedData.eventId}}"
  }
 ]
}
```

Workflow 2: Purchase Event (Payment Completion)

2.1 Create Second Zap

1. Create new Zap
2. Name: "GHL Payment → Facebook Purchase"

2.2 Set Up Trigger - Webhooks by Zapier

Choose Trigger App: Webhooks by Zapier
Event: Catch Hook
Webhook URL: Copy the provided URL (different from first workflow)

2.3 Configure Second GoHighLevel Webhook

In GoHighLevel:
1. Create webhook: "Purchase Event"
2. **Trigger:** Pipeline stage change to "Paid in Full"
3. **URL:** Paste second Zapier webhook URL

2.4 Add Filter Step for Purchase

Filter Conditions (ALL must be true):
- Pipeline stage contains "Paid in Full"
- Facebook Click ID exists
- Email exists
- Payment status = "Paid in Full"
- Revenue amount > 0

2.5 Add Code Step for Purchase Processing

Input Data:
- `email` : Contact email
- `phone` : Contact phone
- `firstName` : Contact first name
- `lastName` : Contact last name
- `facebookClickId` : Facebook Click ID
- `contactId` : GoHighLevel contact ID
- `revenueAmount` : Payment amount
- `productName` : Product/service name
- `transactionId` : Payment transaction ID
- `pixelId` : Your Facebook Pixel ID

JavaScript Code:

```
 const crypto = require(\'crypto\');

function hashData(data) {
  if (!data) return null;
  return
crypto.createHash(\'sha256\').update(data.toLowerCase().trim()).digest(\'hex\');
}

function normalizePhone(phone) {
  if (!phone) return null;
  let normalized = phone.replace(/\D/g, \'\');
  if (normalized.length === 10) {
```

```
    normalized = \'1\' + normalized;
  }
  return normalized;
}

function formatFacebookClickId(fbclid) {
  if (!fbclid) return null;

  if (fbclid.startsWith(\'fb.1.\')) {
    return fbclid;
  }

  const timestamp = Math.floor(Date.now() / 1000);
  return `fb.1.${timestamp}.${fbclid}`;
}

// Process the input data
const processedData = {
  hashedEmail: hashData(inputData.email),
  hashedPhone: hashData(normalizePhone(inputData.phone)),
  hashedFirstName: hashData(inputData.firstName),
  hashedLastName: hashData(inputData.lastName),
  formattedFbc: formatFacebookClickId(inputData.facebookClickId),
  eventTime: Math.floor(Date.now() / 1000),
  eventId: `purchase_${inputData.contactId}_${inputData.transactionId}_$
{Date.now()}`,
  pixelId: inputData.pixelId,
  revenueAmount: parseFloat(inputData.revenueAmount) || 0,
  productName: inputData.productName || \'Service Purchase\',
  transactionId: inputData.transactionId || `txn_${Date.now()}`,
  originalEmail: inputData.email,
  originalPhone: inputData.phone
};

// Validate required data
if (!processedData.hashedEmail || !processedData.formattedFbc) {
  throw new Error(\'Missing required data: email or Facebook Click ID\');
}

if (processedData.revenueAmount <= 0) {
  throw new Error(\'Invalid revenue amount\');
}

output = [{processedData}];
```

2.6 Add HTTP Request for Purchase Event

URL: `https://graph.facebook.com/v18.0/{{processedData.pixelId}}/events`

Method: POST

Headers:

```
{
 "Content-Type": "application/json",
 "Authorization": "Bearer YOUR_FACEBOOK_ACCESS_TOKEN"
}
```

Data (JSON):

```
{
 "data": [
  {
   "event_name": "Purchase",
   "event_time": "{{processedData.eventTime}}",
   "action_source": "website",
   "event_source_url": "https://yoursite.com/payment",
   "user_data": {
    "em": ["{{processedData.hashedEmail}}"],
    "ph": ["{{processedData.hashedPhone}}"],
    "fn": ["{{processedData.hashedFirstName}}"],
    "ln": ["{{processedData.hashedLastName}}"],
    "fbc": "{{processedData.formattedFbc}}"
   },
   "custom_data": {
    "content_name": "{{processedData.productName}}",
    "content_category": "Service Purchase",
    "value": "{{processedData.revenueAmount}}",
    "currency": "USD",
    "content_ids": ["{{processedData.transactionId}}"],
    "content_type": "product",
    "num_items": 1
   },
   "event_id": "{{processedData.eventId}}"
  }
 ]
}
```

Error Handling and Monitoring

3.1 Add Success Notifications

For both workflows, add Gmail steps:

Success Notification:
- **Trigger:** HTTP status 200 from Facebook API
- **Action:** Send success email
- **Subject:** "Facebook Conversion Event Sent Successfully"
- **Body:** Include contact details and event type

3.2 Add Error Notifications

Error Notification:
- **Trigger:** HTTP status != 200 from Facebook API
- **Action:** Send error email
- **Subject:** "Facebook Conversion API Error"
- **Body:** Include error details and contact information

3.3 Enhanced Error Handling in Code Steps

```
// Enhanced error handling for both workflows
try {
  // Your data processing code here

  // Validate required fields
  const requiredFields = [\'hashedEmail\', \'formattedFbc\'];
  for (const field of requiredFields) {
    if (!processedData[field]) {
      throw new Error(`Missing required field: ${field}`);
    }
  }

  // Additional validation for Purchase workflow
  if (inputData.workflowType === \'purchase\' && processedData.revenueAmount
<= 0) {
    throw new Error(\'Invalid revenue amount for purchase event\');
  }

  output = [{processedData}];
} catch (error) {
  console.error(\'Data processing error:\', error.message);

  // Send error data for debugging
  output = [{
    error: true,
    message: error.message,
    inputData: inputData,
    timestamp: new Date().toISOString()
  }];
}
```

Advanced Features

4.1 Batch Processing for High Volume

If you have high volume, modify the workflows to batch events:

```
// Collect events over time window
const eventBatch = [];
const batchWindow = 300; // 5 minutes

// Add current event to batch
eventBatch.push(processedData);

// Send batch when full or time window expires
if (eventBatch.length >= 100 || timeWindowExpired) {
  // Send batch to Facebook API
  const batchPayload = {
    data: eventBatch
  };
}
```

4.2 Dynamic Event Values

Implement sophisticated value calculation:

```
// Advanced value calculation for CompleteRegistration
function calculateAdvancedAttendanceValue(inputData) {
  let baseValue = 100;

  // Factor in lead source quality
  if (inputData.leadSource === \'facebook\') baseValue *= 1.2;

  // Factor in survey score
  const surveyScore = parseInt(inputData.surveyScore) || 5;
  baseValue *= (surveyScore / 10);

  // Factor in booking speed
  const bookingSpeed = parseInt(inputData.bookingSpeedHours) || 48;
  if (bookingSpeed < 24) baseValue *= 1.3;
  if (bookingSpeed < 6) baseValue *= 1.5;

  // Factor in event type
  const eventType = inputData.eventType || \'standard\';
  const eventMultipliers = {
    \'standard\': 1.0,
    \'premium\': 1.5,
    \'vip\': 2.0,
    \'enterprise\': 3.0
  };
  baseValue *= eventMultipliers[eventType] || 1.0;

  return Math.round(baseValue * 100) / 100; // Round to 2 decimal places
}
```

4.3 A/B Testing Event Values

Test different value assignments:

```
// A/B test different event values
const testGroup = inputData.contactId % 2; // Simple A/B split

let eventValue;
if (testGroup === 0) {
  // Group A: Conservative values
  eventValue = 100;
} else {
  // Group B: Aggressive values
  eventValue = 150;
}

// Track which group for analysis
processedData.testGroup = testGroup === 0 ? \'conservative\' : \'aggressive\';
processedData.eventValue = eventValue;
```

Testing and Validation

5.1 End-to-End Testing Protocol

Test CompleteRegistration Workflow:
1. Create test contact in GoHighLevel
2. Add Facebook Click ID and event details
3. Move to "Attended Event - Qualified Lead" stage
4. Verify Zapier triggers correctly
5. Check Facebook Events Manager for event

Test Purchase Workflow:
1. Use same test contact
2. Add revenue amount and payment details
3. Move to "Paid in Full" stage
4. Verify Zapier triggers correctly
5. Check Facebook Events Manager for purchase event

5.2 Data Quality Validation

- [ ] Both workflows trigger independently
- [ ] Event IDs are unique between workflows
- [ ] Revenue amounts are accurate for Purchase events

- [ ] Attribution data matches between events
- [ ] Error handling works for both workflows

5.3 Performance Monitoring

- [ ] Monitor Zapier task usage for both workflows
- [ ] Track success rates for each event type
- [ ] Monitor Facebook API response times
- [ ] Validate event deduplication works correctly

This dual-workflow setup ensures accurate tracking of both attendance (CompleteRegistration) and payment (Purchase) events, giving Facebook the data it needs to optimize for your most valuable outcomes.